SIMATS ENGINEERING

Department of Information Technology

DSA01 – Object-Oriented Programming with C++

ASSIGNMENT 2 – CO2

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO2: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Student Name	I. Lahari	**Register Number**	192472183
Department / Section	CSE-AI	**Academic Year / Semester**	3 year
Faculty Name	Amirtha. S	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	12 -11 - 2026	**Date of Evaluation**	
GitHub Repository	DSA0115-<192472183>-Assignment2	**Compiler**	g++ 13.3.0, -std=c++17 -Wall -Wextra

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Input and Output	4		

Task	Task Title	Max	Marks Awarded	Remarks
9	Implementation and Testing	4		
10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original Design	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	50		Percentage:

OVERALL REMARKS

I. lahari		
Student Signature	**Faculty Signature**	**Program Director**

OBJECT-ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 2 – CO2

CO2 Coverage: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 12/10/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

1. Create a GitHub repository named DSA01-<RegisterNumber>-Assignment2 and share it with the course faculty.
2. Keep one folder for each task (Q1, Q2 … Q7), containing C++ source files, input files and output screenshots.
3. Include a README.md with name, register number, compiler/version, exact g++ command, run steps and a short result note for every task.
4. Commit work progressively, with at least one commit for each task.
5. Add the GitHub link to this report and submit the working repository link.

GitHub Repository Link: https://github.com/lahari1426/DSA0115_192472183.git

How to Answer the Additional Questions

Complete the Code: Replace every TODO with working code without changing the rest of the supplied program, execute it and compare the output with the expected output.

Predict the Output: Write the exact output before running the program. Then execute it and explain differences using the relevant C++ rules.

Alternative Concept and Complexity Analysis: Explain the alternative design and compare time and space complexity using Big-O notation. Add actual measurements only after running the programs on your computer.

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

1. Explanation and Three Implementations

Call by value creates a local copy of the argument. Changes made to that copy do not change the original variable. Call by reference gives the function an alias for the caller's variable, while call by pointer passes its address and changes the original value by dereferencing the pointer.

```
#include <iostream>
using namespace std;

void byValue(long long fee, int percent) {
```

```
    fee -= fee * percent / 100;
}
void byReference(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
void byPointer(long long* fee, int percent) {
    if (fee != nullptr)
        *fee -= (*fee) * percent / 100;
}
int main() {
    long long a = 45000, b = 45000, c = 45000;
    byValue(a, 10);
    byReference(b, 10);
    byPointer(&c, 10);
    cout << a << " " << b << " " << c << '\\n';
}
```

Expected output: 45000 40500 40500

Only the reference and pointer versions change the caller's fee. The value version changes only its local copy. A const reference should be used when a function must read a large object without copying it and must not modify it.

2. Predict the Output

```
#include <iostream>
void change(int a, int& b, int* c) {
    a += 10; b += 10; *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) {
    std::cout << x + y + z << ' ';
}
int main() {
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\\n';
    show(1); show(1, 2); show(1, 2, 3);
    std::cout << '\\n';
}
```

Predicted and actual output:

```
1 12 13\n1 100\n16 13 6
```

Explanation: a remains 1 because it is passed by value. b and c become 12 and 13 because they are passed by reference and pointer. bigger(a, c) returns a reference to c because c is larger, so the assignment changes c to 100. The show() calls use the defaults (y=5, z=10), then (z=10), then no defaults.

3. Complete the Code

```
void applyConcession(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
```

Expected output:

after call by value : 45000\nafter call by reference: 40500

The call by value leaves the original fee at 45000 because the function receives a separate copy.

4. Alternative Concept and Complexity

A return-value design can be written as: long long applyConcession(long long fee, int percent) { return fee - fee * percent / 100; }. The caller uses fee = applyConcession(fee, 10). All versions take O(1) time. A long long value copy uses constant space (typically 8 bytes); a reference or pointer passes an address rather than copying the object. Passing a large Student object by value copies all its members, while const reference avoids that copy. For several concessions, returning the new fee is clear and avoids hidden mutation; reference is useful when in-place update is the required interface.

Q2 – Return by Reference and Its Dangers (4 Marks)

1. Highest Mark by Reference

```
#include <iostream>
using namespace std;

int& highest(int marks[], int n) {
    int index = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[index]) index = i;
    return marks[index];
}

// WRONG: never return a reference to a local variable.
// int& badReference() {
//     int value = 50;
//     return value; // compiler warning: reference to local variable returned
// }

int main() {
    int marks[] = {72, 91, 84, 88};
    highest(marks, 4) += 2;
    for (int m : marks) cout << m << ' ';
    cout << '\\n';
}
```

Output: 72 93 84 88

The function returns an alias to the actual array element, so the caller can modify it and use the function call on the left side of an assignment. Returning a local variable is unsafe because the local object is destroyed when the function ends; the returned reference dangles and using it causes undefined behaviour.

2. Reference, Const Reference and Copy

- int&: caller can read and modify the original object.
- const int&: caller can read the original object but cannot modify it through that reference.
- int: returns an independent copy; changing the copy does not change the array.

If a std::vector grows and reallocates its storage, references and pointers to its elements may become invalid. An index remains usable after growth if the logical element order is preserved, but the index must still be checked against the current size.

3. Alternative: Index or Pointer

```
int highestIndex(const int a[], int n) {
    int k = 0;
    for (int i = 1; i < n; ++i)
        if (a[i] > a[k]) k = i;
    return k;
}
// Use: marks[highestIndex(marks, 4)] += 2;
// Pointer alternative: int* p = &marks[highestIndex(marks, 4)]; *p += 2;
```

Finding the maximum takes O(n) time and O(1) extra space in all three approaches. An index is generally safer across vector reallocation because it can be used again to access the new storage. A pointer or reference may become invalid after reallocation.

Q3 – Default Arguments and Function Overloading (6 Marks)

1. Why the Original Declarations Are Ambiguous

The declarations charge(double), charge(double,bool), and charge(double,bool=false,bool=false) overlap. A call such as charge(2.0) can match all three declarations because the later parameters have defaults. The compiler cannot select one unique best overload, so the call is ambiguous.

2. Redesigned Overloads

```
#include <iostream>
using namespace std;

struct Parcel {
    double weightKg;
    bool express;
    bool insured;
};

double charge(double weightKg, bool express = false, bool insured = false) {
    double amount = weightKg * 40.0;
    if (express) amount += 20.0;
    if (insured) amount += 10.0;
    return amount;
}
double charge(int weightGrams) {
    return (weightGrams / 1000.0) * 40.0;
}
double charge(const Parcel& p) {
    return charge(p.weightKg, p.express, p.insured);
}
int main() {
    Parcel p{2.5, true, true};
    cout << charge(2) << '\\n';
    cout << charge(2.0f) << '\\n';
```

```
    cout << charge(2.0, true) << '\\n';
    cout << charge(2.0, false, true) << '\\n';
    cout << charge(1500) << '\\n';
    cout << charge(p) << '\\n';
}
```

Illustrative selection predictions: charge(2) selects the int overload (exact match); charge(2.0f) selects the double overload by floating-point promotion; charge(2.0,true) and charge(2.0,false,true) select the double overload; charge(1500) selects the int overload; charge(p) selects the Parcel overload.

The example uses a simple illustrative rate of Rs 40 per kg, with fixed optional additions. The rate is not supplied by the assignment and can be changed to the rate specified by the faculty.

3. Overload Resolution and Defaults

The compiler prefers an exact match, then a promotion, then a standard conversion. Default arguments fill missing trailing parameters only after an overload is selected. They must be supplied from right to left: a declaration such as f(int a=1, int b) is invalid because a non-default parameter follows a default parameter.

4. Alternative Concept and Complexity

A single Parcel structure with fields for weight, express and insurance can replace several overloads. A function template is another option for compatible weight types. Overload resolution occurs at compile time and adds no runtime cost. Each function call is O(1) for this calculation. A Parcel passed by value copies its fields; const Parcel& avoids the copy. A structure is easy to extend with more options, while overloads make input types explicit.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

1. Savings Account Class

```
#include <iostream>
using namespace std;

class SavingsAccount {
private:
    long long accountNumber;
    double balance;
    double transactions[10];
    int count;
    int next;

public:
    SavingsAccount(long long number, double opening)
        : accountNumber(number), balance(opening), count(0), next(0) {}

    void deposit(double amount);
    bool withdraw(double amount);
    void miniStatement() const;
};

void SavingsAccount::deposit(double amount) {
```

```
    if (amount <= 0) return;
    balance += amount;
    transactions[next] = amount;
    next = (next + 1) % 10;
    if (count < 10) ++count;
}
bool SavingsAccount::withdraw(double amount) {
    if (amount <= 0 || amount > balance) return false;
    balance -= amount;
    transactions[next] = -amount;
    next = (next + 1) % 10;
    if (count < 10) ++count;
    return true;
}
void SavingsAccount::miniStatement() const {
    cout << "Account: " << accountNumber
         << " Balance: " << balance << '\\n';
    int start = (next - count + 10) % 10;
    for (int i = 0; i < count; ++i)
        cout << transactions[(start + i) % 10] << '\\n';
}
int main() {
    SavingsAccount a(1001, 1000);
    a.deposit(500);
    a.withdraw(200);
    a.deposit(100);
    a.miniStatement();
}
```

The account number, balance, transaction array and counters are private so outside code cannot change them directly. Public member functions control deposits, withdrawals and statement display. Some functions are declared inside the class and defined outside using the scope resolution operator.

2. Handling the 11th Transaction

With shifting, the program moves transactions 1–9 one position to the left and stores the new transaction at the end. This takes O(k) time for k stored entries. With a circular index, the next transaction overwrites the oldest entry and the index wraps around; insertion takes O(1) time. Both use O(10) fixed storage. For 50 transactions, the array size and modulus change from 10 to 50, so the circular-buffer design requires only a small capacity change.

3. Alternative: std::deque

A deque can use push_back(newTransaction) followed by pop_front() when its size exceeds 10. These operations are O(1) at the ends and the storage is O(k), where k is at most 10. A fixed circular array avoids dynamic allocation and has predictable memory; a deque is easier to resize and maintain.

Q5 – Static Members and Arrays of Objects (7 Marks)

1. Static Ticket Counter

```
#include <iostream>
```

```
#include <string>
using namespace std;

class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    string name;

public:
    explicit BusTicket(const string& n) : seat(0), name(n) {
        if (issued < CAPACITY) seat = ++issued;
    }
    static int count() { return issued; }
    void print() const {
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat) : string("Waiting"))
             << '\\n';
    }
};
int BusTicket::issued = 0;

int main() {
    BusTicket trip[4] = {
        BusTicket("Asha"), BusTicket("Bala"),
        BusTicket("Chitra"), BusTicket("Dev")
    };
    for (const BusTicket& t : trip) t.print();
    cout << "Issued: " << BusTicket::count() << '\\n';
}
```

Expected output:

```
Asha: Seat 1\nBala: Seat 2\nChitra: Seat 3\nDev: Waiting\nIssued: 3
```

The static variable issued belongs to the class and is shared by all ticket objects. It is declared inside the class and defined once outside the class. The static function count() can access static data, but it has no this pointer and cannot directly access an individual object’s non-static members.

2. Predict the Output

```
class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};
int Token::next = 100;
int main() {
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' '
              << counter[2].get() << ' ' << extra.get()
```

```
        << ' ' << Token::issued() << '\\n';
}
```

Predicted output: 101 103 104 4. The static counter starts at 100. Three array objects and one extra object are constructed, so the constructor runs four times and increments the shared counter to 104.

3. Creating Tickets in a Function

```
void createTrip() {
    BusTicket tickets[2] = {BusTicket("Ravi"), BusTicket("Maya")};
    for (const BusTicket& t : tickets) t.print();
    cout << "Total issued: " << BusTicket::count() << '\\n';
}
// Calling createTrip() twice does not reset the static counter.
// The second call continues from the count left by the first call.
```

If the capacity is shared globally, the second call continues the same count. If separate buses need independent seat counters, a TicketOffice object or one counter per bus is more suitable.

4. Alternative Concept and Complexity

A global counter has O(1) time and one counter's storage, but any function can modify it. A TicketOffice object passed to each ticket constructor also uses O(1) time and one counter per office, and it supports two buses at the same time. The static member is convenient when the count genuinely belongs to one class-wide ticket pool; TicketOffice gives better separation for multiple buses.

Q6 – Friend Functions and Inline Functions (8 Marks)

1. Completed Friend Functions

```
#include <iostream>
using namespace std;

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

inline int minutesBetween(const Time& a, const Time& b) {
    int x = a.h * 60 + a.m;
    int y = b.h * 60 + b.m;
    return (x > y) ? x - y : y - x;
}
inline bool earlier(const Time& a, const Time& b) {
    return a.h * 60 + a.m < b.h * 60 + b.m;
}
int main() {
    Time departure(9, 45), arrival(13, 10);
    cout << minutesBetween(departure, arrival) << ' '
         << boolalpha << earlier(arrival, departure) << '\\n';
```

```
}
```
Expected output: 205 false

The friend functions are not members of Time, so they are called like ordinary functions. They can access private members because the class explicitly grants friendship. A friend is useful when an operation treats two Time objects equally. Making h and m public would weaken encapsulation.

2. Correctness Tests

```
Time t1(11, 30), t2(11, 30);
Time beforeNoon(11, 55), afterNoon(12, 10);
cout << minutesBetween(t1, t2) << ' ' << earlier(t1, t2) << '\\n';
cout << minutesBetween(beforeNoon, afterNoon) << ' '
    << earlier(beforeNoon, afterNoon) << '\\n';
```
Expected results: 0 0 for equal times, and 15 1 for 11:55 compared with 12:10.

3. Inline Meaning and Alternative Design

The inline keyword requests that the compiler consider replacing a function call with the function body. It does not guarantee that substitution will happen. Modern compilers make this decision using optimisation information. Small functions defined inside a class are implicitly inline.

The same logic can be implemented as member functions such as a.minutesBetween(b) and a.isEarlier(b), or through public getters. All designs perform O(1) work per call. Friend functions keep the comparison symmetric and avoid exposing getters solely for this operation. Member functions can express the operation from the perspective of the first object and may provide tighter encapsulation.

Timing investigation: compile the test with -O0 and -O2, run a large number of calls, and record the measured time on the submission computer. Do not copy invented timings; optimisation level, compiler and hardware affect the result.

```
g++ -std=c++17 -O0 time_test.cpp -o time_O0\n./time_O0\ng++ -std=c++17 -O2 time_test.cpp -
o time_O2\n./time_O2
```

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected Program: Q4 – Mini-Statement Transaction Storage

A. Diagnose the Limitation (2 Marks)

The original shifting-array approach becomes inefficient when a new transaction arrives after the array is full. To retain only the latest k transactions, it must move up to k−1 elements on every insertion. This makes each full-buffer insertion O(k), and repeated insertions require repeated shifting.

B. Define the Design Objective (2 Marks)

Redesign transaction storage so that adding a transaction takes O(1) time while retaining only the most recent 10 (or 50) transactions and preserving chronological display order.

C. Design the Revised Solution (4 Marks)

Use a circular buffer with a fixed array, a next index and a count. The next index identifies the slot to overwrite. After inserting a transaction, advance the index using modulo capacity. The design needs no dynamic allocation and uses O(k) fixed memory.

```
// Circular-buffer insertion logic
transactions[next] = newAmount;
next = (next + 1) % CAPACITY;
if (count < CAPACITY) ++count;

// Display starts at the oldest retained transaction
int start = (next - count + CAPACITY) % CAPACITY;
for (int i = 0; i < count; ++i)
   print(transactions[(start + i) % CAPACITY]);
```

D. Predict Before Implementing (1 Mark)

For a full buffer of 10 entries, the original shifting approach performs up to 9 moves for each new entry. The circular buffer should perform one write and one index update, independent of the number of retained entries. Both should display the same latest 10 transactions in chronological order.

E. Implement and Validate (2 Marks)

Test	**Shifting array**	**Circular buffer**
Insert first 10 transactions	Stores all 10	Stores all 10
Insert transaction 11	Moves up to 9 entries, then inserts	Overwrites oldest slot; advances index
Display after 11 inserts	Transactions 2–11	Transactions 2–11
Insertion time	O(k) when full	O(1)

Validation procedure: insert transaction values 1 through 11, print the mini-statement, and verify that the output contains 2 through 11 in that order. Repeat with 51 transactions when capacity is 50. Record actual execution time only after running both implementations with the same input.

F. Defend the Final Design (2 Marks)

The circular buffer meets the objective because insertion does not shift existing elements. Its trade-off is that the display order requires index calculation and the implementation is slightly more complex. A shifting array may still be suitable for a very small number of transactions when simplicity is more important than insertion speed. For a 50-entry or frequently updated mini-statement, the circular buffer is the better design.

Submission Checklist

- Create folders Q1 through Q7 in the GitHub repository.
- Save each completed C++ program as a separate .cpp file in its task folder.
- Compile with: g++ -std=c++17 -Wall -Wextra filename.cpp -o program
- Run every program and save terminal output screenshots.

- Write each output prediction before execution and retain the prediction in the report.
- For Q6 and Q7, record actual timing measurements from your own computer.
- Add README.md and commit each task separately.